

Assignment Two

Database Development using SQL Server

Task 5: Report Compilation

Karen Lau (--)

EECS 3421: Introduction to Database Systems

—

November 7, 2025

1. TASK 1: Table Creation

Four tables were created, and the SQL queries used to create each table are shown in [Figures 1-4](#) in the appendix. The **tables** were created in the following **order** to maintain referential integrity:

- **JOB**

- JOB_CODE *CHAR(3)*
- JOB_DESCRIPTION *VARCHAR(40)*
- JOB_CHG_HOUR *NUMERIC(6,2)*
- JOB_LAST_UPDATE *DATE*

- **EMPLOYEE**

- EMP_NUM *CHAR(3)*
- EMP_LNAME *VARCHAR(20)*
- EMP_FNAME *VARCHAR(20)*
- EMP_INITIAL *CHAR(1); (accepts) Null*
- EMP_HIREDATE *DATE*
- JOB_CODE (**FK**) *CHAR(3); (accepts) Null*

- **PROJECT**

- PROJ_NUM *CHAR(2)*
- PROJ_NAME *VARCHAR(50)*
- PROJ_VALUE *NUMERIC(10,2) **
- PROJ_BALANCE *NUMERIC(10,2) **
- EMP_NUM (**FK**) *CHAR(3); (accepts) Null*

- **ASSIGNMENT**

- ASSIGN_NUM *CHAR(4)*
- ASSIGN_DATE *DATE*
- PROJ_NUM (**FK**) *CHAR(2)*
- EMP_NUM (**FK**) *CHAR(3)*
- ASSIGN_JOB *CHAR(3)*
- ASSIGN_CHG_HR *NUMERIC(6,2)*
- ASSIGN_HOURS *NUMERIC(3,1) ***
- ASSIGN_CHARGE *NUMERIC(6,2)*

* Sample data had values with only 9 digits, specified 10 for some leniency.

** Max hours in a day is 24.0, so max digits is 3.

The above **ASSIGNMENT** table was created according to the relational schema provided in the Assignment 2 manual (page 2). That is, no alterations were made to further normalise the table into 3NF. Additionally, unless otherwise specified, all columns in each table were defined with the 'NOT NULL' constraint, as the sample datasets provided did not include empty cell values.

2. TASK 2: Integrity Constraints and Relationships

An important quality of any database is ensuring that its data remains reliable, consistent and accurate – this is known as data integrity. To manage data integrity in databases, we consider two main principles: entity integrity and referential integrity.

Entity integrity ensures that every record in a database can be uniquely identified. This is accomplished by defining a non-null, unique primary key for every relation. Thus, for this project, all relations – **JOB**, **EMPLOYEE**, **PROJECT** and **ASSIGNMENT** – have explicitly defined primary keys with a NOT NULL constraint, as shown in [Figures 1-4](#) of the appendix.

Referential integrity ensures the validity and consistency of relationships and data between tables. This is maintained by monitoring foreign keys by conserving insertion integrity, update integrity, and deletion integrity.

- For this project, we created the tables (and inserted data) in an order that respected insertion integrity. More specifically, tables without foreign keys dependencies, such as the **JOB** table, were created first. Then, dependent tables such as **EMPLOYEE**, **PROJECT** and **ASSIGNMENT** were created – each containing foreign keys that referenced columns that had already been defined in previously created tables. Furthermore, all data that was inserted into the dependent tables contained valid foreign keys that matched primary keys in the parent tables.
- To uphold update and deletion integrity, appropriate constraints were defined for each foreign key in the database relations.
 - **EMPLOYEE TABLE: JOB_CODE foreign key**
 - ON DELETE SET NULL: When a job code is deleted, this indicates that the role is no longer available in the database. However, I assumed that the employees who previously held this position would not be removed from the database or considered terminated. Instead, their JOB_CODE value is set to NULL, indicating that their role information is temporarily undefined until they are updated or reassigned.
 - ON UPDATE CASCADE: When a job code is updated, the update should be automatically applied to all employees that hold that position.
 - **PROJECT TABLE: EMP_NUM foreign key**
 - ON DELETE SET NULL: When an employee (project manager) is deleted from the database, their associated projects should remain, since these projects (I assume) belong to the company rather than the individual manager. Consequently, the EMP_NUM value is set to NULL, indicating that the project temporarily does not have an assigned manager.

- **ON UPDATE CASCADE:** When an employee number is updated, the update should be automatically applied to all projects that the individual is managing.
- **ASSIGNMENT TABLE: EMP_NUM and PROJ_NUM foreign keys**
The EMP_NUM foreign key is considered the dominant cascading path in the **ASSIGNMENT** table. The Employee relation was chosen as the dominant path since every assignment fundamentally depends on an employee. The data in the assignment table primarily describes the work that employee performs on a project, rather than the information about the projects themselves.
- **EMP_NUM ON DELETE CASCADE:** If an employee is deleted from the database, any assignment records associated with that employee should also be removed. Once the employee no longer exists in the system, their related project assignments are meaningless.
- **EMP_NUM ON UPDATE CASCADE:** When an employee number is updated, the update should automatically apply to all their project assignments.
- **PROJ_NUM ON DELETE NO ACTION:** Ideally, this foreign key would also use 'on delete cascade', since assignment records referencing a deleted project are meaningless. However, MSSQL restricts multiple cascading paths within the same table to prevent unintended mass deletions. To preserve the intended behaviour, a trigger ([Figure 5](#) in appendix) was implemented to mimic this cascade deletion effect. The trigger ensures that when a project is deleted, all corresponding assignment records are also removed.
- **PROJ_NUM ON UPDATE NO ACTION:** Similarly, this foreign key would ideally use 'on update cascade', so any valid project number updates are automatically reflected in the assignment table. Again, since MSSQL restricts multiple cascading paths, this functionality is mimicked by creating a trigger ([Figure 6](#) in appendix). The trigger will update all related assignment records when a project number is updated.

3. TASK 3: Enter Sample Data for Tables

In Task 1, four tables were created: **JOB**, **EMPLOYEE**, **PROJECT**, and **ASSIGNMENT**. The SQL queries used to insert sample data into each table, can be found as [figures 7-10](#) in the appendix. Data was inserted in the following order – **JOB** > **EMPLOYEE** > **PROJECT** > **ASSIGNMENT** – to maintain referential integrity. I inserted the sample data provided on pages 3 and 4 of the Assignment 2 manual,

while also including a personalized record into the EMPLOYEE, and ASSIGNMENT tables, as suggested ([figure 11](#) in appendix). Below I have included screenshots of each of the tables with inserted data

JOB Table

Queried: **SELECT * FROM JOB**

	JOB_CODE	JOB_DESCRIPTION	JOB_CHG_HOUR	JOB_LAST_UPDATE
1	500	Programmer	35.75	2017-11-20
2	501	Systems Analyst	96.75	2017-11-20
3	502	Database Designer	125.00	2018-03-21
4	503	Electrical Engineer	84.50	2017-11-20
5	504	Mechanical Engineer	67.90	2017-11-20
6	505	Civil Engineer	55.78	2017-11-20
7	506	Clerical Support	26.87	2017-11-20
8	507	DSS Analyst	45.95	2017-11-20
9	508	Applications Designer	48.10	2018-03-21
10	509	Bio Technician	34.55	2017-11-20
11	510	General Support	18.36	2017-11-20

EMPLOYEE Table

Queried: **SELECT * FROM EMPLOYEE**

	EMP_NUM	EMP_LNAME	EMP_FNAME	EMP_INITIAL	EMP_HIREDATE	JOB_CODE
1	101	News	John	G	2000-11-08	502
2	102	Senior	David	H	1989-07-12	501
3	103	Arbough	June	E	1996-12-01	500
4	104	Ramoras	Anne	K	1987-11-15	501
5	105	Johnston	Alice	K	1993-02-01	502
6	106	Smithfield	William	NULL	2004-06-22	500
7	107	Alonzo	Maria	D	1993-10-10	500
8	108	Washington	Ralph	B	1991-08-22	501
9	109	Smith	Larry	W	1997-07-18	501
10	110	Olenko	Gerald	A	1995-12-11	505
11	111	Wabash	Geoff	B	1991-04-04	506
12	112	Smithson	Darlene	M	1994-10-23	507
13	113	Joebrood	Delbert	K	1996-11-15	508
14	114	Jones	Annelise	NULL	1993-08-20	508
15	115	Bawangi	Travis	B	1992-01-25	501
16	116	Pratt	Gerald	L	1997-03-05	510
17	117	Williamson	Angie	H	1996-06-19	509
18	118	Frommer	James	J	2005-01-04	510
19	220	Lau	Karen	S	2025-11-07	507

PROJECT Table

Queried: **SELECT * FROM PROJECT**

	PROJ_NUM	PROJ_NAME	PROJ_VALUE	PROJ_BALANCE	EMP_NUM
1	15	Evergreen	1453500.00	1002350.00	103
2	18	Amber Wave	3500500.00	2110346.00	108
3	22	Rolling Tide	805000.00	500345.20	102
4	25	Star flight	2650500.00	2309880.00	107

ASSIGNMENT Table

Queried: **SELECT * FROM ASSIGNMENT**

	ASSIGN_NUM	ASSIGN_DATE	PROJ_NUM	EMP_NUM	ASSIGN_JOB	ASSIGN_CHG_HR	ASSIGN_HOURS	ASSIGN_CHARGE
3	1003	2018-03-23	18	117	509	34.55	2.0	69.10
4	1004	2018-03-23	18	103	500	35.75	5.9	210.93
5	1005	2018-03-23	25	108	501	96.75	2.2	212.85
6	1006	2018-03-23	22	104	501	96.75	4.2	406.35
7	1007	2018-03-23	25	113	508	48.10	3.8	182.78
8	1008	2018-03-24	18	103	500	35.75	0.9	32.18
9	1009	2018-03-24	15	115	501	96.75	5.6	541.80
10	1010	2018-03-24	15	117	509	34.55	2.4	82.92
11	1011	2018-03-24	25	105	502	125.00	4.3	537.50
12	1012	2018-03-24	18	108	501	96.75	3.4	328.95
13	1013	2018-03-25	25	115	501	96.75	2.0	193.50
14	1014	2018-03-25	22	104	501	96.75	2.8	270.90
15	1015	2018-03-25	15	103	500	35.75	6.1	218.08
16	1016	2018-03-25	22	105	502	125.00	4.7	587.50
17	1017	2018-03-25	18	117	509	34.55	3.8	131.29
18	1018	2018-03-26	25	117	509	34.55	2.2	76.01
19	1019	2018-03-26	25	104	501	96.75	4.9	474.08
20	1020	2018-03-26	15	101	502	125.00	3.1	387.50
21	1021	2018-03-26	22	108	501	96.75	2.7	261.23
22	1022	2018-03-26	22	115	501	96.75	4.9	474.08
23	1023	2018-03-26	22	105	502	125.00	3.5	437.50
24	1024	2018-03-26	15	103	500	35.75	3.3	117.98
25	1025	2018-03-27	18	117	509	34.55	4.2	145.11
26	1026	2025-11-07	25	220	507	45.95	4.0	183.80

4. TASK 4: Design Queries for Information Search

The SQL-queries and outputs for each of the requested information searches can be found below:

- List all employees who were hired **before January 1, 1995**

```
1 SELECT * FROM EMPLOYEE WHERE EMP_HIREDATE < '01-Jan-1995';
```

EMP_NUM	EMP_LNAME	EMP_FNAME	EMP_INITIAL	EMP_HIREDATE	JOB_CODE
102	Senior	David	H	1989-07-12	501
104	Ramoras	Anne	K	1987-11-15	501
105	Johnston	Alice	K	1993-02-01	502
107	Alonzo	Maria	D	1993-10-10	500
108	Washington	Ralph	B	1991-08-22	501
111	Wabash	Geoff	B	1991-04-04	506
112	Smithson	Darlene	M	1994-10-23	507
114	Jones	Annelise	NULL	1993-08-20	508
115	Bawangi	Travis	B	1992-01-25	501

- List the **job code, job description**, and **employee last name and first name**. Sort the result by **job description**, then by **employee last name**.

```
1 SELECT E.JOB_CODE, JOB_DESCRIPTION, EMP_LNAME, EMP_FNAME
2 FROM EMPLOYEE as E, JOB as J
3 WHERE E.JOB_CODE = J.JOB_CODE
4 ORDER BY JOB_DESCRIPTION;
```

JOB_CODE	JOB_DESCRIPTION	EMP_LNAME	EMP_FNAME
508	Applications Designer	Joenbrood	Delbert
508	Applications Designer	Jones	Annelise
509	Bio Technician	Williamson	Angie
505	Civil Engineer	Olenko	Gerald
506	Clerical Support	Wabash	Geoff
502	Database Designer	News	John
502	Database Designer	Johnston	Alice
507	DSS Analyst	Smithson	Darlene
507	DSS Analyst	Lau	Karen
510	General Support	Pratt	Gerald
510	General Support	Frommer	James
500	Programmer	Arbough	June
500	Programmer	Smithfield	William
500	Programmer	Alonzo	Maria
501	Systems Analyst	Senior	David
501	Systems Analyst	Ramoras	Anne
501	Systems Analyst	Washington	Ralph
501	Systems Analyst	Smith	Larry
501	Systems Analyst	Bawangi	Travis

- Ordered by Job Description

```

1 SELECT E.JOB_CODE, JOB_DESCRIPTION, EMP_LNAME, EMP_FNAME
2 FROM EMPLOYEE as E, JOB as J
3 WHERE E.JOB_CODE = J.JOB_CODE
4 ORDER BY EMP_LNAME;

```

	JOB_CODE	JOB_DESCRIPTION	EMP_LNAME	EMP_FNAME
1	500	Programmer	Alonzo	Maria
2	500	Programmer	Arbough	June
3	501	Systems Analyst	Bawangi	Travis
4	510	General Support	Frommer	James
5	508	Applications Designer	Joenbrood	Delbert
6	502	Database Designer	Johnston	Alice
7	508	Applications Designer	Jones	Annelise
8	507	DSS Analyst	Lau	Karen
9	502	Database Designer	News	John
10	505	Civil Engineer	Olenko	Gerald
11	510	General Support	Pratt	Gerald
12	501	Systems Analyst	Ramoras	Anne
13	501	Systems Analyst	Senior	David
14	501	Systems Analyst	Smith	Larry
15	500	Programmer	Smithfield	William
16	507	DSS Analyst	Smithson	Darlene
17	506	Clerical Support	Wabash	Geoff
18	501	Systems Analyst	Washington	Ralph
19	509	Bio Technician	Williamson	Angie

- Ordered by Employee Last Name

- c. List the **first and last names** of all employees assigned to the project named “Evergreen”.

```

1 SELECT E.EMP_FNAME, E.EMP_LNAME
2 FROM EMPLOYEE as E, ASSIGNMENT as A, PROJECT as P
3 WHERE E.EMP_NUM = A.EMP_NUM
4 AND P.PROJ_NUM = A.PROJ_NUM
5 AND P.PROJ_NAME = 'Evergreen';

```

	EMP_FNAME	EMP_LNAME
1	Travis	Bawangi
2	Angie	Williamson
3	June	Arbough
4	John	News
5	June	Arbough

*June Arbough shows up twice in the result since she appears in the **Assignment** table with *Assign_Num* 1015 and 1024.

- d. For each project, list the **project name**, the **number of employees** worked for the project, the **total number of hours worked**, and the **total charges** assigned to each project. Sort the result by project name.

```

1 SELECT P.PROJ_NAME, COUNT(DISTINCT A.EMP_NUM) as TOTAL_EMPLOYEES,
2 SUM(A.ASSIGN_HOURS) as TOTAL_HOURS,
3 SUM(A.ASSIGN_CHARGE) as TOTAL_CHARGE
4 FROM ASSIGNMENT as A, PROJECT as P
5 WHERE P.PROJ_NUM = A.PROJ_NUM
6 GROUP BY P.PROJ_NAME, P.PROJ_NUM
7 ORDER BY P.PROJ_NAME;

```

PROBLEMS

5

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

QUERY RESULTS

Results (1)

Messages

	PROJ_NAME	TOTAL_EMPLOYEES	TOTAL_HOURS	TOTAL_CHARGE
1	Amber Wave	3	23.7	1042.69
2	Evergreen	4	20.5	1348.28
3	Rolling Tide	5	27.0	2582.67
4	Star flight	7	23.4	1860.52

- e. List all employees who have been assigned to **more than one project**. Show their EMP_NUM, EMP_LNAME, EMP_FNAME, and the number of distinct projects they were assigned to.

1	SELECT E.EMP_NUM, E.EMP_LNAME, E.EMP_FNAME, COUNT(DISTINCT A.PROJ_NUM) as NUM_PROJECTS
2	FROM EMPLOYEE as E, ASSIGNMENT as A, PROJECT as P
3	WHERE E.EMP_NUM = P.EMP_NUM
4	GROUP BY E.EMP_NUM, E.EMP_FNAME, E.EMP_LNAME
5	HAVING COUNT(DISTINCT A.PROJ_NUM) >1;

PROBLEMS	3	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS	QUERY RESULTS
----------	---	--------	---------------	----------	-------	---------------

Results (1)		Messages		Open
	EMP_NUM	EMP_LNAME	EMP_FNAME	NUM_PROJECTS
1	102	Senior	David	4
2	103	Arbough	June	4
3	107	Alonzo	Maria	4
4	108	Washington	Ralph	4

Appendix

**For this assignment I ran my queries (MSSQL) in VS Code which was connected to the Azure SQL database*

Figure 1: SQL Code - JOB Table Creation

```
CREATE TABLE JOB(  
  JOB_CODE CHAR(3) NOT NULL PRIMARY KEY,  
  JOB_DESCRIPTION VARCHAR(40) NOT NULL,  
  JOB_CHG_HOUR NUMERIC(6,2) NOT NULL,  
  JOB_LAST_UPDATE DATE NOT NULL  
);
```

Figure 2: SQL Code - EMPLOYEE Table Creation

```
CREATE TABLE EMPLOYEE(  
  EMP_NUM CHAR(3) NOT NULL PRIMARY KEY,  
  EMP_LNAME VARCHAR(20) NOT NULL,  
  EMP_FNAME VARCHAR(20) NOT NULL,  
  EMP_INITIAL CHAR(1) NULL,  
  EMP_HIREDATE DATE NOT NULL,  
  JOB_CODE CHAR(3) NULL,  
  CONSTRAINT FK_JOB_CODE FOREIGN KEY(JOB_CODE) REFERENCES JOB(JOB_CODE)  
  ON DELETE SET NULL -- IF DELETE JOB TITLE - SETS EMPLOYEE JOB CODE TO NULL  
  ON UPDATE CASCADE -- IF UPDATE JOB CODE, UPDATES FOR ALL EMPLOYEES  
);
```

Figure 3: SQL Code - PROJECT Table Creation

```
CREATE TABLE PROJECT (  
  PROJ_NUM CHAR(2) NOT NULL PRIMARY KEY,  
  PROJ_NAME VARCHAR(50) NOT NULL,  
  PROJ_VALUE NUMERIC(10,2) NOT NULL,  
  PROJ_BALANCE NUMERIC(10,2) NOT NULL,  
  EMP_NUM CHAR(3) NULL,  
  CONSTRAINT FKP_EMP_NUM FOREIGN KEY (EMP_NUM) REFERENCES EMPLOYEE(EMP_NUM)  
  ON DELETE SET NULL -- DELETE EMPLOYEE; STILL HAVE PROJECT - JUST NO ONE WORKING ON IT  
  ON UPDATE CASCADE -- IF EMPLOYEE NUMBER CHANGES, SHOULD UPDAE ON PROJECT  
);
```

Figure 4: SQL Code - ASSIGNMENT Table Creation

```
CREATE TABLE ASSIGNMENT (  
  ASSIGN_NUM CHAR(4) NOT NULL PRIMARY KEY,  
  ASSIGN_DATE DATE NOT NULL,  
  PROJ_NUM CHAR(2) NOT NULL,  
  EMP_NUM CHAR(3) NOT NULL,  
  ASSIGN_JOB CHAR(3) NOT NULL,  
  ASSIGN_CHG_HR NUMERIC(6,2) NOT NULL,  
  ASSIGN_HOURS NUMERIC(3,1) NOT NULL,  
  ASSIGN_CHARGE NUMERIC(6,2) NOT NULL,  
  CONSTRAINT FKA_EMP_NUM FOREIGN KEY (EMP_NUM) REFERENCES EMPLOYEE(EMP_NUM)  
  ON DELETE CASCADE -- IF EMPLOYEE IS DELETED, SHOULD BE DELETED FROM ASSIGNMENTS (DOMINANT)  
  ON UPDATE CASCADE, -- IF EMPLOYEE NUM IS UPDATED, SHOULD BE UPDATED ON ALL ASSIGNMENTS  
  CONSTRAINT FK_PROJ_NUM FOREIGN KEY (PROJ_NUM) REFERENCES PROJECT(PROJ_NUM)  
  ON DELETE NO ACTION -- unable to have multiple cascades - ideally delete assignments if projects are deleted; use trigger  
  ON UPDATE NO ACTION-- unable to have multiple cascades - ideally update assignments if project num updates; use trigger  
);
```

Figure 5: SQL Code - ASSIGNMENT Table Delete Trigger Creation

A delete trigger was created to bypass the 'multiple cascades' error that SQL threw. This trigger is created for the PROJ_NUM Foreign key to mimic an 'ON DELETE CASCADE' constraint. This trigger method was recommended as a solution by TA Mohammed. Since we hadn't covered trigger creation in class, I researched examples online and implemented the code based on what I found.

```
1 CREATE TRIGGER trg_Project_Delete
2 ON PROJECT
3 AFTER DELETE
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7     DELETE FROM ASSIGNMENT
8     WHERE PROJ_NUM IN (SELECT PROJ_NUM FROM DELETED);
9 END;
10 GO
11
```

Figure 6: SQL Code - ASSIGNMENT Table Update Trigger Creation

An update trigger was created to bypass the 'multiple cascades' error that SQL threw. This trigger is created for the PROJ_NUM Foreign key to mimic an 'ON UPDATE CASCADE' constraint. Similarly, I researched examples of triggers online and implemented the code based on what I found.

```
CREATE TRIGGER trg_Project_Update
ON PROJECT
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF UPDATE(PROJ_NUM)
    BEGIN
        UPDATE A
        SET A.PROJ_NUM = I.PROJ_NUM
        FROM ASSIGNMENT AS A
        INNER JOIN DELETED AS D ON A.PROJ_NUM = D.PROJ_NUM
        INNER JOIN INSERTED AS I ON D.PROJ_NUM = I.PROJ_NUM;
    END;
END;
GO
```

Figure 7: SQL Code - JOB Table Insert Data Queries

```
INSERT INTO JOB VALUES ('500', 'Programmer', 35.75, '20-Nov-2017'),
('501', 'Systems Analyst', 96.75, '20-Nov-2017'),
('502', 'Database Designer', 125.00, '21-Mar-2018'),
('503', 'Electrical Engineer', 84.50, '20-Nov-2017'),
('504', 'Mechanical Engineer', 67.90, '20-Nov-2017'),
('505', 'Civil Engineer', 55.78, '20-Nov-2017'),
('506', 'Clerical Support', 26.87, '20-Nov-2017'),
('507', 'DSS Analyst', 45.95, '20-Nov-2017'),
('508', 'Applications Designer', 48.10, '21-Mar-2018'),
('509', 'Bio Technician', 34.55, '20-Nov-2017'),
('510', 'General Support', 18.36, '20-Nov-2017');
```

Figure 8: SQL Code - EMPLOYEE Table Insert Data Queries

```
INSERT INTO EMPLOYEE VALUES ('101', 'News', 'John', 'G', '08-Nov-2000', '502'),
('102', 'Senior', 'David', 'H', '12-Jul-1989', '501'),
('103', 'Arbough', 'June', 'E', '01-Dec-1996', '500'),
('104', 'Ramoras', 'Anne', 'K', '15-Nov-1987', '501'),
('105', 'Johnston', 'Alice', 'K', '01-Feb-1993', '502'),
('106', 'Smithfield', 'William', NULL, '22-Jun-2004', '500'),
('107', 'Alonzo', 'Maria', 'D', '10-Oct-1993', '500'),
('108', 'Washington', 'Ralph', 'B', '22-Aug-1991', '501'),
('109', 'Smith', 'Larry', 'W', '18-Jul-1997', '501'),
('110', 'Olenko', 'Gerald', 'A', '11-Dec-1995', '505'),
('111', 'Wabash', 'Geoff', 'B', '04-Apr-1991', '506'),
('112', 'Smithson', 'Darlene', 'M', '23-Oct-1994', '507'),
('113', 'Joebrood', 'Delbert', 'K', '15-Nov-1996', '508'),
('114', 'Jones', 'Annelise', NULL, '20-Aug-1993', '508'),
('115', 'Bawangi', 'Travis', 'B', '25-Jan-1992', '501'),
('116', 'Pratt', 'Gerald', 'L', '05-Mar-1997', '510'),
('117', 'Williamson', 'Angie', 'H', '19-Jun-1996', '509'),
('118', 'Frommer', 'James', 'J', '04-Jan-2005', '510');
```

Figure 9: SQL Code - PROJECT Table Insert Data Queries

```
INSERT INTO PROJECT VALUES ('15', 'Evergreen', 1453500.00, 1002350.00, '103'),
('18', 'Amber Wave', 3500500.00, 2110346.00, '108'),
('22', 'Rolling Tide', 805000.00, 500345.00, '102'),
('25', 'Star flight', 2650500.00, 2309880.00, '107');
```

Figure 10: SQL Code - ASSIGNMENT Table Insert Data Queries

```
INSERT INTO ASSIGNMENT VALUES (1001, '22-Mar-2018', 18, 103, 500, 35.75, 3.5, 125.13),
(1002, '22-Mar-2018', '22', '117', '509', 34.55, 4.2, 145.11),
(1003, '23-Mar-2018', '18', '117', '509', 34.55, 2.0, 69.10),
(1004, '23-Mar-2018', '18', '103', '500', 35.75, 5.9, 210.93),
(1005, '23-Mar-2018', '25', '108', '501', 96.75, 2.2, 212.85),
(1006, '23-Mar-2018', '22', '104', '501', 96.75, 4.2, 406.35),
(1007, '23-Mar-2018', '25', '113', '508', 48.10, 3.8, 182.78),
(1008, '24-Mar-2018', '18', '103', '500', 35.75, 0.9, 32.18),
(1009, '24-Mar-2018', '15', '115', '501', 96.75, 5.6, 541.80),
(1010, '24-Mar-2018', '15', '117', '509', 34.55, 2.4, 82.92),
(1011, '24-Mar-2018', '25', '105', '502', 125.00, 4.3, 537.50),
(1012, '24-Mar-2018', '18', '108', '501', 96.75, 3.4, 328.95),
(1013, '25-Mar-2018', '25', '115', '501', 96.75, 2.0, 193.50),
(1014, '25-Mar-2018', '22', '104', '501', 96.75, 2.8, 270.90),
(1015, '25-Mar-2018', '15', '103', '500', 35.75, 6.1, 218.08),
(1016, '25-Mar-2018', '22', '105', '502', 125.00, 4.7, 587.50),
(1017, '25-Mar-2018', '18', '117', '509', 34.55, 3.8, 131.29),
(1018, '26-Mar-2018', '25', '117', '509', 34.55, 2.2, 76.01),
(1019, '26-Mar-2018', '25', '104', '501', 96.75, 4.9, 474.08),
(1020, '26-Mar-2018', '15', '101', '502', 125.00, 3.1, 387.50),
(1021, '26-Mar-2018', '22', '108', '501', 96.75, 2.7, 261.23),
(1022, '26-Mar-2018', '22', '115', '501', 96.75, 4.9, 474.08),
(1023, '26-Mar-2018', '22', '105', '502', 125.00, 3.5, 437.50),
(1024, '26-Mar-2018', '15', '103', '500', 35.75, 3.3, 117.98),
(1025, '27-Mar-2018', '18', '117', '509', 34.55, 4.2, 145.11);
```

Figure 11: SQL Code - Insert Personalized Record Data Queries

```
INSERT INTO EMPLOYEE VALUES ('220', 'Lau', 'Karen', 'S', '7-Nov-2025', 507); -- Job: DSS Analyst
INSERT INTO ASSIGNMENT VALUES ('1026', '7-Nov-2025', '25', '220', '507', 45.95, 4.0, 183.80); -- Project: Star Flight
```